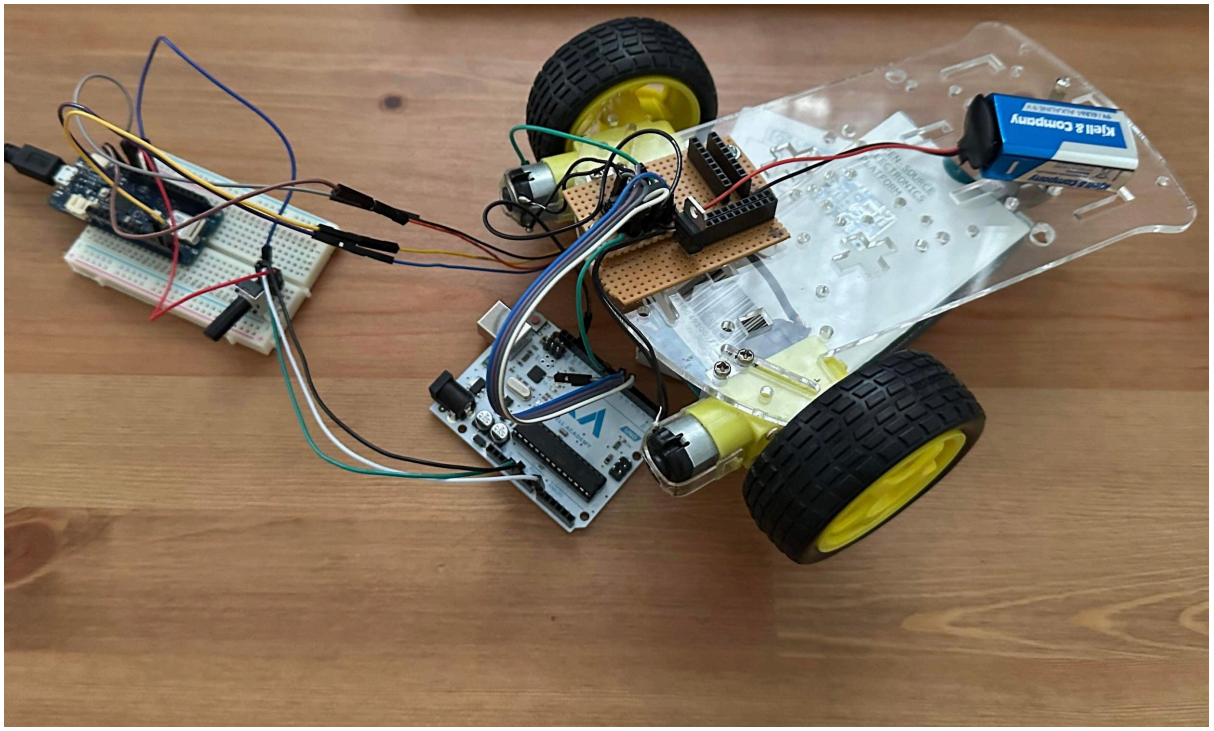
PWM-kontrollerad likströmsmotor med encoder

Projektrapport

Loke Lindberg Nilsson, Oscar Olheden, Isabella Bergström,
Emilia Lindqvist, Sasithorn Bystedt, Jegors Liteplo



Sammanfattning

Denna rapport presenterar en prototyp av en PWM-kontrollerad likströmsmotor med en optisk roterande encoder för mätning och reglering av hastighet och positionering. Genom användning av Arduino IoT-cloud skapas ett användargränssnitt för övervakning och visualisering av encoderdata i realtid, med fokus på teknisk precision och användarvänlighet. Rapporten beskriver även tekniska aspekter såsom kopplingen mellan potentiometer, motor och Arduino UNO samt behov och nytta av den tillämpade tekniska lösningen.

Innehållsförteckning

Inledning **1**
Problemformulering **1**
Kravspecifikation 1
Skallkrav 1
Börkrav 1
Teknisk beskrivning av produkt **2**
Konstruktion 2
Encoder 2
Motorkontroller 2
IoT-cloud 3
Komponentlista 3
Resultat **3**
Diskussion **4**
Slutsats **4**
Referenser
Bilagor

Inledning

I det framåtskridande teknologiska samhället introduceras ständigt nya innovationer för att möta samhällets behov. Encodern, en central komponent för mätning och reglering av rotationshastighet och rotationsvinkel, har spelat en avgörande roll i tekniska framsteg. Detta projekt fokuserar på att lösa problemet med insamling och presentation av rotationsdata genom användning av en optisk encoder, Arduino och IoT-cloud för realtidsvisualisering.

Problemformulering

Mätning och reglering av rotationshastighet och rotationsvinkel är grundläggande för att kunna optimera säkerheten och prestandan i diverse maskiner. Denna tekniska lösning kan användas inom en mängd olika områden och i en rad olika maskiner, stora som små, från 3D-skrivare och drönare till bilar och medicinsk utrustning. Problemet som vår prototyp är tänkt att lösa är insamling och presentation av den nödvändiga informationen om rotationen av en axel och dess sammanhängande parametrar.

Kravspecifikation

<u>Skallkrav</u>

- Avläsa encoder med Arduino MKR.
- Anslut Arduino MKR till internet via WiFi
- Avläs encoder över internet, i realtid.
- Presentation av sensordata grafiskt.
- Snurra encoder med motor.
- Styra motor med potentiometer.

<u>Börkrav</u>

- Styra motor i båda riktningarna.

Teknisk beskrivning av produkt

Lösningen består av en encoder som sitter på en roterande axel. Encodern läses av med hjälp av en Arduino MKR 1010 WiFi som framöver kommer att refereras som Arduino MKR. Arduino MKR registrerar rotation och rotationshastighet på encoderns axel och med hjälp av IoT-cloud, som är uppkopplat via Wi-Fi till Arduino MKR, kan data läsas av i realtid. För att kunna testa och demonstrera lösningen används en likströmsmotor med tillhörande motorkontroller bestående av en Arduino UNO, H-brygga och potentiometer.

Konstruktion

Systemet utgörs av två delar. En första del för avläsning av vinkel och rotationshastighet av en axel och för att ladda upp samma information till IoT Cloud. En andra del för att sätta en rotation på den axel som läses av i den första delen. Den första delen består mer ingående av Arduino MKR och Encoder. Den andra delen består i sin tur mer ingående av likströmsmotor, 9-volts batteri, Arduino UNO, H-brygga och potentiometer. En förtydligande figur återfinns under bilaga 1.

Encoder

Den encoder som används i projektet är av typen optisk encoder. Encodern använder två ljussensorer och två ljuskällor som är exakt kalibrerade i förhållande till en hålad skiva (Leach, 2014). Encodern används i systemet genom att möjliggöra exakt avläsning och uppföljning av motoraxelns rotation. När motorn driver rotationen av skivan registrerar ljussensorerna förändringar i ljusnivån när hålen passerar. Denna information används för att beräkna rotationshastighet och riktning för motoraxeln. Då hålens positioner runt skivans rand också är kända (skivans omkrets och hålens distans från varandra), kan rotationens hastighet och riktning på encoderns axel beräknas.

Motorkontroller

Motorn styrs av en H-brygga, reglerad av en Arduino UNO och en potentiometer, där H-bryggan möjliggör både medurs och moturs rotation. Genom att styra switcharna i H-bryggan kan systemet ändra strömmens riktning genom motorn, vilket i sin tur styr rotationsriktningen. För att styra motorns hastighet används pulsviddsmodulering, redan inbyggd som Arduino-funktionen *analogWrite(pin, pwmValue)*. Genom ett antal matematiska funktioner och konverteringar kan potentiometern styra både H-bryggan och pulsviddsmoduleringen.

IoT-cloud

Arduino IoT-cloud används för att övervaka och visualisera data från en encoder och motor. Via IoT-clouds inbyggda verktyg som dashboards och widgets skapas då ett

användargränssnitt. Två mätare *Angle* och *Angular velocity* införs för att ge en detaljerad övervakning och analys av systemets prestanda i realtid. Se bilaga 2. För att visualisera enhetens prestanda över tid införs även två tabeller, som presenterar realtidsdatan från mätarna *Angle* och *Angular velocity* över en tidsaxel.

För att åstadkomma önskat användargränssnitt körs en C++ kod där data från Arduino MKR omarbetas till lämpligt format. Se bilaga 3. I koden skapas två objekt av klasserna CloudAngle och CloudVelocity. Dessa används för att koppla variabler i koden till motsvarande variabler på respektive mätare.

CloudAngle angle, synkad med mätare *Angle*;

- Skapar ett objekt av typen CloudAngle. Det representerar en variabel som är synkroniserad med en mätare för vinklar på IoT-cloudens instrumentbräda.

CloudVelocity velocity, synkad med mätare *Angular velocity*;

- Här skapas ett objekt av typen CloudVelocity. Det representerar en variabel som är synkroniserad med en mätare för vinkelhastighet på IoT-cloudens instrumentbräda.

Objekten angle och velocity kopplar den lokala koden och dess representation på IoT-cloud:s användargränssnitt.

Komponentlista

- Likströmsmotor: *Pololu Micro Metal Gearmotor HP 1000 6V*
- Optisk encoder: *Pololu Optical Encoder 2590 5V*
- H-brygga: *DRV8833 Dual Motor Driver Carrier*
- Arduino MKR
- Arduino UNO
- Potentiometer
- Batteri 9 volt
- Breadboard
- Kablar

Resultat

För att utvärdera prototypens prestanda genomfördes följande tester:

Enkel rotationshastighetsmätning:

- Test: Prototypens motor roterades med en konstant hastighet.
- Resultat: Encoderdata överfördes och visades grafiskt i IoT-cloud. Mätaren för Angular velocity visade en konstant rotationshastighet.

Styrning av motorhastighet med potentiometer:

- Test: Potentiometern justerades för att ändra motorhastigheten.
- Resultat: Mätaren för Angular velocity reagerade i realtid på potentiometerns justeringar, vilket visade korrekt överföring och styrning av motorhastigheten.

Fjärravläsning över internet:

- Test: IoT-cloud anslöts från en extern plats för fjärravläsning av rotationsdata.
- Resultat: Encoderdata överfördes över Wi-Fi och var tillgänglig för fjärravläsning, vilket bekräftade pålitlig internetanslutning och dataöverföring.

Motorstyrning i båda riktningar:

- Test: Riktningen på rotationen ändrades genom att manipulera potentiometern.
- Resultat: Prototypen kan styra motorrotationen i både klockvis och moturs riktning beroende på potentiometerns justering.

Resultaten av dessa tester bekräftar att prototypen uppfyller kraven enligt kravspecifikationen och fungerar i olika scenarier. Prototypen visar sig vara pålitlig och användarvänlig för övervakning och styrning av rotationsdata.

Diskussion

Användning av C++-kod och IoT-clouds grafiska programmering har förenklat skapandet av gränssnitt. Fokus har lagts på att samla in och överföra data utan komplexiteten av en tvåvägskommunikation, vilket resulterar i en stabil lösning när övervakning och insamling av data är det primära målet. Med de erhållna resultaten kan några användningsområden för lösningen konstateras. Den tekniska lösningen erbjuder bred användbarhet inom olika områden trots sin specifika funktionalitet genom att möjliggöra övervakning och visualisering av motorprestanda. Lösningen tillåter förbättrade styrningsmöjligheter snarare än aktiv styrning i system där motorer är en central komponent. Inom industriell automation kan lösningen användas för att övervaka och styra maskiner och motorer i produktionsanläggningar, medan den inom robotik möjliggör exakt styrning och positionering av motorer. Inom fordonsteknik kan lösningen tillämpas för att optimera prestanda i elektriska bilar eller drönare. Även inom smart hem-teknik öppnas möjligheter för styrning och övervakning av motoriserade enheter som persienner eller grindar.

Slutsats

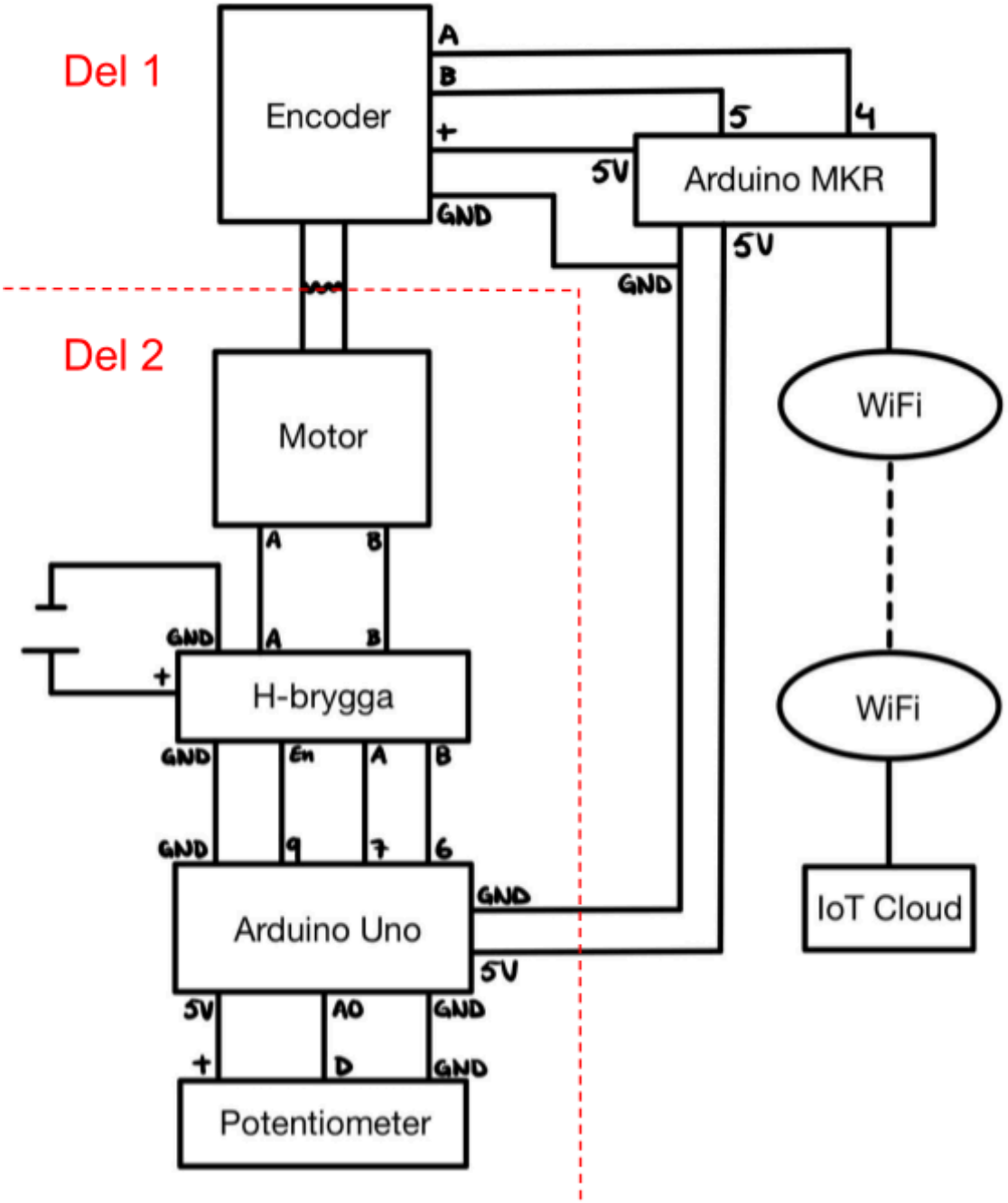
Slutsatsen kan dras från prövning av prototyp; det går att läsa av data från en encoder med hjälp av en Arduino MKR 1010 Wifi. Datan som laddas upp från mikrokontrollern till ett moln kan även analyseras och visualiseras i realtid

Referenser

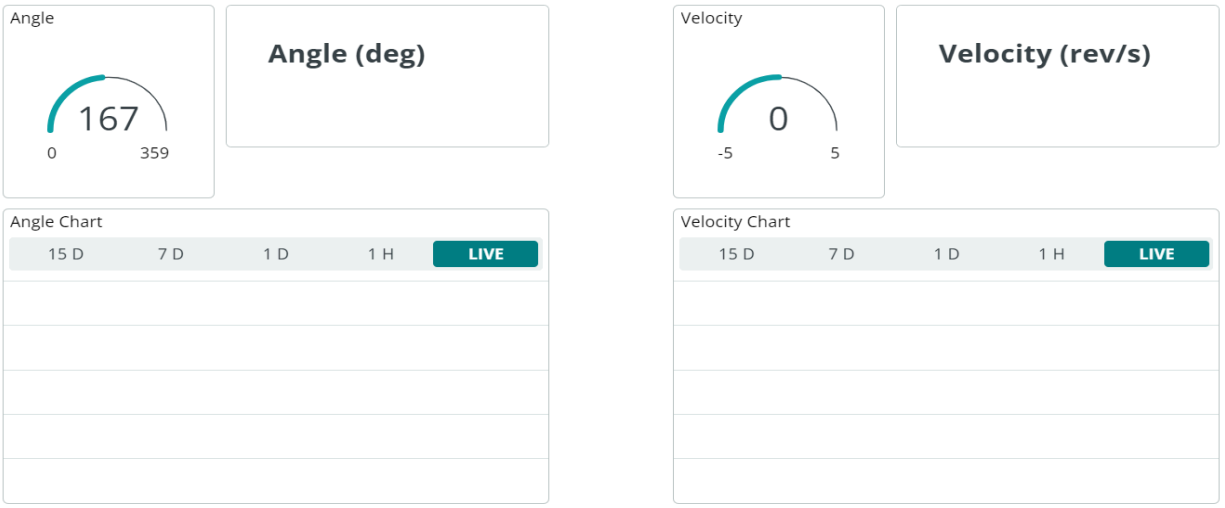
Leach, Richard. 2014. *Fundamental Principles of Engineering Nanometrology*. Tillgängligt via https://www.sciencedirect.com/topics/engineering/optical-encoders [Hämtad 2023-12-22]

Bilagor

Bilaga 1. Kretsschema



Bilaga 2. Paneler i IoT-cloud



Bilaga 3 *MF1001_Motor_nov23a.ino*

```
/*
  Sketch generated by the Arduino IoT Cloud Thing "Untitled"

https://create.arduino.cc/cloud/things/4f16867f-f660-4844-8b98-a3d6e9180450

  Arduino IoT Cloud Variables description

  The following variables are automatically generated and updated when
changes are made to the Thing

  CloudAngle angle;
  CloudVelocity velocity;

  Variables which are marked as READ/WRITE in the Cloud Thing will also have
functions
  which are called when their values are changed from the Dashboard.
  These functions are generated with the Thing and added at the end of this
sketch.
*/

#include "thingProperties.h"

//Encoder
#ifndef ENCODER_H
#define ENCODER_H

//Encoder.h
class Encoder {
  public:
    int pinA;
    int pinB;
    int lastB;
    int counter;
    int countsPerRev;
    Encoder(int pinA, int pinB, int countsPerRevolution);
    void begin();
    float getRevolutions();
    float getAngle();
    float getAngularVelocity();
    void update();
};

//Encoder.cpp
Encoder::Encoder(int pinA, int pinB, int countsPerRevolution) {
  this->pinA = pinA;
```

```
  this->pinB = pinB;
  this->countsPerRev = countsPerRevolution;
}

void Encoder::begin() {
  pinMode(pinA, INPUT);
  pinMode(pinB, INPUT);

  this->counter = 0;
  this->lastB = digitalRead(pinB);
}

float Encoder::getRevolutions() {
  return (float)counter / countsPerRev;
}

void Encoder::update() {
  int bState = digitalRead(pinB);
  if (lastB != bState) {
    if (digitalRead(pinB) != digitalRead(pinA)) counter += 1;
    else counter -= 1;
    lastB = bState;
  }
}
#endif


#define INTERVAL 0.1
#define ENCODER_B_PIN 5

Encoder encoder(4, ENCODER_B_PIN, 600);
int lastMillis = 0;
int lastStep = 0;
float speed = 0;

void updateEncoders() {
  encoder.update();
}

void setup() {
  // Initialize serial and wait for port to open:
  Serial.begin(256000);
  // This delay gives the chance to wait for a Serial Monitor without
blocking if none is found
  delay(1500);

  // Defined in thingProperties.h
```

```
  initProperties();

  // Connect to Arduino IoT Cloud
  ArduinoCloud.begin(ArduinoIoTPreferredConnection);

  /*
     The following function allows you to obtain more information
     related to the state of network and IoT Cloud connection and errors
     the higher number the more granular information you'll get.
     The default is 0 (only errors).
     Maximum is 4
  */
  setDebugMessageLevel(2);
  ArduinoCloud.printDebugInfo();

  encoder.begin();
  attachInterrupt(ENCODER_B_PIN, updateEncoders, CHANGE);
}

void loop() {
  ArduinoCloud.update();


  float deltaSeconds = (millis() - lastMillis) / 1000.0;

  if (deltaSeconds >= INTERVAL) {
    speed = (float)(encoder.counter - lastStep) / deltaSeconds;

    lastStep = encoder.counter;
    lastMillis = millis();
  }


  /*
      Serial.print("Steps: ");
      Serial.print(encoder.counter);
      Serial.print("   Revolutions: ");
      Serial.print(encoder.getRevolutions());
      Serial.print("   Speed: ");*/
  Serial.println(speed / encoder.countsPerRev);
  //Serial.println(" revs/s");

  velocity = speed / encoder.countsPerRev;


  int angleDegrees = (int)floor(360 * (float)encoder.counter/
encoder.countsPerRev) % 360;
```

```
  if(angleDegrees < 0){
    angleDegrees += 360;
  }

  angle = angleDegrees;
}
```